LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

PRAKTIKUM 6

Dzakiy Ibnu Khoir (5225600061)

PROGRAM STUDI SARJANA TERAPAN TEKNOLOGI GAME

JURUSAN TEKNOLGOI MULTEMDIA KREATIF

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2026

Latihan 1

```
#include <iostream>
using namespace std;
struct Node
{
    char data;
    Node *left;
    Node *right;
};
Node *createNode(char data)
{
    Node *node = new Node();
    node->data = data;
    node->left = NULL;
    node->right = NULL;
    return node;
}

int main()
{
    // Membuat node
    Node *A = createNode('A');
    Node *B = createNode('B');
    Node *C = createNode('C');
    // Menghubungkan node
    A->left = B;
    A->right = C;
    // Output
    cout << "Root : " << A->data << endl;
    cout << "Left Child : " << A->left->data << endl;
    cout << "Right Child : " << A->right->data << endl;
    return 0;
}
```

1. Apa fungsi pointer left dan right pada node?

Pointer left dan right berfungsi untuk “mendeklarasikan?” node child yang akan ada, pada contoh di atas, parent adalah A, kemudian B berada di kiri(anak) dari A begitu juga C

2. Mengapa struktur tree membutuhkan pointer?

Sepemahaman saya dari gemini, pointer digunakan karena lokasi memori pada node tidak tepat berada di sebelah dari parent(tidak seperti array), jadi membutuhkan pointer untuk menghubungkannya, mirip

seperti linked list. Dan juga ketika kita ingin reparent sebuah node, kita tinggal arahkan pointer ke parent yang baru

Percobaan 2

```
#include <iostream>
#include <vector>
using namespace std;
// =====================
// STRUCT NODE
// =====================
struct Node
{
    string data;
    vector<Node *> children;
};
// =====================
// CREATE NODE
// =====================
Node *createNode(string data)
{
    Node *node = new Node();
    node->data = data;
    return node;
}
// =====================
// TAMBAH CHILD
// =====================
void addChild(Node *parent, Node *child)
{
    parent->children.push_back(child);
}
// =====================
// TAMPILKAN TREE (RECURSIVE)
// =====================
void printTree(Node *root, int level = 0)
{
    if (root == NULL)
        return;
    // indentasi biar terlihat tree
    for (int i = 0; i < level; i++)
    {
        cout << "--";
```

```
    }
    cout << root->data << endl;
    // tampilkan semua child
    for (int i = 0; i < root->children.size(); i++)
    {
        printTree(root->children[i], level + 1);
    }
}
// ====================
// MAIN
// ====================
int main()
{
    // root
    Node *fire = createNode("Fire");
    // child
    Node *fireball = createNode("Fireball");
    Node *flame = createNode("Flame Burst");
    Node *meteor = createNode("Meteor");
    // hubungan
    addChild(fire, fireball);
    addChild(fire, flame);
    addChild(fire, meteor);
    // tampilkan tree
    cout << "Skill Tree:\n";
    printTree(fire);
    return 0;
}
```

1. Gambarkan bentuk struktur tree pada contoh diatas ?

Pada code di atas, fungsi addchild memiliki 2 parameter (parent, child), jadi kita bisa dapatkan sebuah visualisasi sederhana dari code di atas. Parent dari semua node yang ditambahkan pada main adalah **Fire**, kemudian node fire memiliki 3 child, yaitu fireball, flame dan meteor

2. Apa yang anda ketahui tentang rekursif? Dan Mengapa butuh rekursif di atas ?

Rekursif adalah fungsi yang memanggil dirinya sendiri, seperti contoh pada printTree, ia memanggil dirinya lagi pada for loop, rekursif dibutuhkan karena digunakan untuk mencari child dari node paling atas(node yang dimasukan dalam parameter di main)